COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	30/07/26	**DATE SUBMITTED:**	30/07/26
NAME:	Mike Justin A. Cabangon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO.: 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A INPUT

```cpp
#include<iostream>
#include<stack> //calling stack from the STL

int main (){
    //create an object for the stacl
    std::stack<int> newStack;

    std::cout<<"Stack Empty?"<<newStack.empty()<<std::endl;

    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty?"<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the elment is: "<<newStack.top()<<std::endl;
    std::cout<<"Removing element from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the elment is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the elment is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the elment is: "<<newStack.top()<<std::endl;
    std::cout<<"Stack Empty?"<<newStack.empty()<<std::endl;


    return 0;
}
```

ILO A OUTPUT

```
Stack Empty?1
Stack Empty?0
The size of the stack: 3
The top of the elment is: 15
Removing element from the stack:
The top of the elment is: 8
The top of the elment is: 3
The top of the elment is: 0
Stack Empty?1
```

ILO B INPUT

```
#include<iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData, choice;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();

int main(){
    std::cout<< "Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(true){
    std::cout<<"stack operations"<<std::endl;
    std::cout<<"1. PUSH\n2.POP\n3.TOP\n4.isEMPTY"<<std::endl;
    std::cin>>choice;
        switch(choice){
            case 1:push();
            break;
            case 2:pop();
            break;
```

```
                case 3:Top();
                break;
                case 4: std::cout<< isEmpty()<<std::endl;
                break;
                default: std::cout<<"Invalid choice."<<std::endl;
            }
        }
        return 0;
    }

    bool isEmpty(){
        //create a condition to check wheter the top element is equal to -1
        if( top == -1) return true;
        return false;
    }

    bool ifFull(){
        //check if we reached the maximum capacity of the array
        if(top > i-1) return true;
        return false;
    }

    void push(){
        //check if full -> if yes return an error
    if(top == 100){
        std::cout<<"Stack overflow";
        return;
        }

        std::cout<<"New value to add into the stack"<<std::endl;
        std::cin >> newData;
        //increment the top(index by 1) then assign the new data;
        stack[++top] = newData;

    }

    void Top(){
        //check if the stack is empty
        if(isEmpty()){
            std::cout<<"Stack underflow"<<std::endl;
        }
        std::cout<<"The element on the top of the stack is: "<<stack[top]<<std::endl;
    }

    void pop(){
        //check if the stack is empty
        if(isEmpty()){
        std::cout<<"Stack underflow"<<std::endl;
        }
        //display the top value
        std::cout<<"Popping"<<stack[top]<<std::endl;
        top--;
    }
```

ILO B OUTPUT

```
1. PUSH
2.POP
3.TOP
4.isEMPTY
1
New value to add into the stack
1
stack operations
1. PUSH
2.POP
3.TOP
4.isEMPTY
2
Popping1
stack operations
1. PUSH
2.POP
3.TOP
4.isEMPTY
3
Stack underflow
The element on the top of the stack is: 0
stack operations
1. PUSH
2.POP
3.TOP
4.isEMPTY
4
1
stack operations
1. PUSH
2.POP
3.TOP
4.isEMPTY
```

Modified ILO B Input: Displaying stacked elements

```cpp
#include<iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData, choice;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void ShowAll();

int main(){
    std::cout<< "Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(true){
    std::cout<<"stack operations"<<std::endl;
    std::cout<<"1.PUSH\n2.POP\n3.TOP\n4.isEMPTY\n5.All elements"<<std::endl;
    std::cin>>choice;
        switch(choice){
            case 1:push();
            break;
            case 2:pop();
            break;
            case 3:Top();
            break;
            case 4:std::cout<< isEmpty()<<std::endl;
            break;
```

```
31                  case 5:ShowAll();
32                  break;
33                  default: std::cout<<"Invalid choice."<<std::endl;
34              }
35          }
36          return 0;
37      }
38
39      bool isEmpty(){
40          //create a condition to check wheter the top element is equal to -1
41          if( top == -1) return true;
42          return false;
43      }
44
45      bool ifFull(){
46          //check if we reached the maximum capacity of the array
47          if(top > i-1) return true;
48          return false;
49      }
50
51      void push(){
52          //check if full -> if yes return an error
53      if(top == 100){
54          std::cout<<"Stack overflow";
55          return;
56          }
57
58          std::cout<<"New value to add into the stack"<<std::endl;
59          std::cin >> newData;
60          //increment the top(index by 1) then assign the new data;
```

```
62
63  }
64
65  void Top(){
66      //check if the stack is empty
67      if(isEmpty()){
68          std::cout<<"Stack underflow"<<std::endl;
69      }
70      std::cout<<"The element on the top of the stack is: "<<stack[top]<<std::endl;
71  }
72
73  void pop(){
74      //check if the stack is empty
75      if(isEmpty()){
76      std::cout<<"Stack underflow"<<std::endl;
77      }
78      //display the top value
79      std::cout<<"Popping"<<stack[top]<<std::endl;
80      top--;
81  }
82
83  void ShowAll(){
84      if(isEmpty()){
85      std::cout<<"Stack underflow"<<std::endl;
86      }
87
88      for(int i = top; i >= 0; i--){
89
90          std::cout<< stack[i] <<std::endl;
91      }
92
93  }
```

Modified ILO B Output: Displaying Stacked elements

```
Enter the number of max elements for the new stack: 2
stack operations
1.PUSH
2.POP
3.TOP
4.isEMPTY
5.All elements
1
New value to add into the stack
3
stack operations
1.PUSH
2.POP
3.TOP
4.isEMPTY
5.All elements
1
New value to add into the stack
4
stack operations
1.PUSH
2.POP
3.TOP
4.isEMPTY
5.All elements
5
4
3
stack operations
1.PUSH
2.POP
3.TOP
4.isEMPTY
5.All elements
```

Modified ILO B.2 main.cpp

```cpp
#include <iostream>
#include "stacklist.h"

int main(){

    std::cout<<"----- PUSHING 'c' 'p' 'e' ONTO THE STACK -----\n";
    std::cout<<"is empty? ";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); //Top should be 'e'
    ShowAll<char>();
    std::cout<<"----- Popping all the elements -----\n";

    while(!isEmpty<char>()){
        std::cout<<"Popped: "<< pop<char>() <<std::endl;

    }

    std::cout<<"\n----- Testing the Underflow -----\n";
    pop<char>();

    return 0;
}
```

Modified ILO B.2 stackList.h

```
#ifndef STACK_LIST_H
#define STACK_LIST_H
#include <iostream>

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;//VALUE THAT NEED TO BE STORED
    Node* next;//POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack
//push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data= newData;

    //new node point to current top, then becomes the new top
    newNode->next= head<T>;
    head<T> = newNode;

}
//pull
```

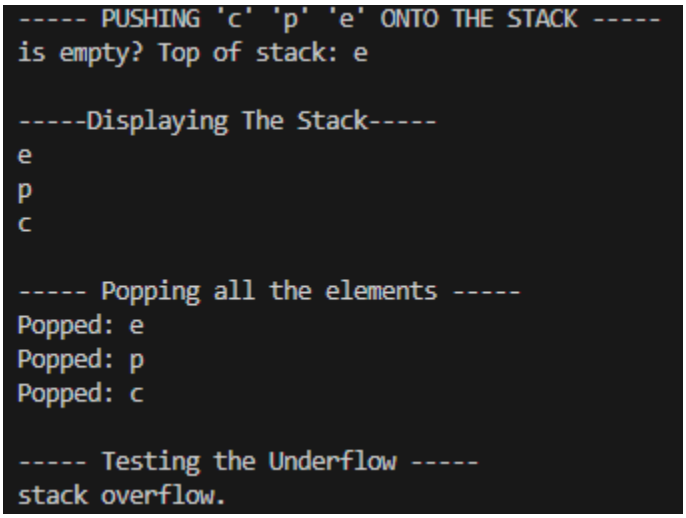
```cpp
template<typename T>
T pop(){
    //check if the stack is empty
    if(head<T> == nullptr){
        std::cout<<"stack overflow. "<<std::endl;

    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;
    //move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed node
    return tempVal;
}

//top
template<typename T>
void Top(){
    //error catching
    if(head<T> == nullptr){
        std::cout<<"stack underflow. "<<std::endl;
    }
    //display the top node
    std::cout<<"Top of stack: "<<head<T>->data <<std::endl;
}
```

```
60  }
61  //isEmpty
62  template<typename T>
63  bool isEmpty(){
64      return head<T> == nullptr;
65  }
66
67  //display all
68  template<typename T>
69  void ShowAll(){
70      if(head<T> == nullptr){
71
72          std::cout << "the stack is empty\n";
73          return;
74      }
75
76      std::cout << "\n-----Displaying The Stack-----\n";
77      Node<T>* currentNode = head<T> ;
78
79      while(currentNode != nullptr){
80
81        std:: cout << currentNode->data << std::endl;
82        currentNode = currentNode-> next;
83
84      }
85      std::cout << "\n";
86
87  }
88
89  #endif
```

Modified ILO B.2 Output


```
----- PUSHING 'c' 'p' 'e' ONTO THE STACK -----
is empty? Top of stack: e

-----Displaying The Stack-----
e
p
c

----- Popping all the elements -----
Popped: e
Popped: p
Popped: c

----- Testing the Underflow -----
stack overflow.
```

Observation
There are different functions executed to insert, pull, and read elements in the stack or to do Stack operations. Stack acts as the last element that goes in is the first element that goes out, simply just like stacking an object in a can. With the modified code, I made a function to read all the elements that are within the stack from the top to bottom by reading each value of the elements in their position.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

IN PROGRESS

ILO C main.cpp

ILO C stackList.h

ILO C OUTPUT

Expression	Valid? (Y/N)	Output(Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D])			

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.